

《人工智能与Python程序设计》——人工智能导论



人工智能与Python程序设计 教研组



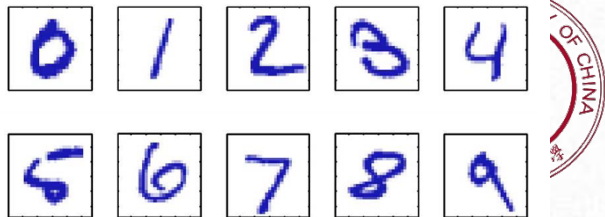
回归与分类：重要概念

- 数据：
 - 一般为空间中点的集合，每个点以 (x_i, y_i) 的形式出现
 - 一般称 x_i 为“特征”， y_i 为对应的“真实值”或“观测值”
- **任务 (task)**：找到由 x 得到 y 的方法，即给定 x ，预测 y
 - 回归 vs 分类
- **模型 (model)**：
 - 猜想 x 到 y 的映射由 $f(x)$ 给出
 - 模型即 $f(x)$ 的具体形式，带有待定参数
- **损失函数 (loss function)**：根据已有数据，评价参数质量的指标
- **训练 (优化) (optimization)**：找到最好的 $f(x)$ 参数的过程，即最小化损失函数的过程
- 预测：给出 x ，计算 $f(x)$ 的过程
- 测试：评价预测结果



回归与分类

- 都是监督学习中常见的**任务**类型
- 区别
 - **任务**类型不同
 - 输出值不同
 - 回归模型一般给出**具体数值**，**输出值连续**
 - 分类模型一般给出**所属类别**，**输出值离散**
 - 进而导致使用的模型不同
- 二分类：
 - 因变量 y 可能属于的两个类分别称为负类和正类，则因变量 $y \in \{0, 1\}$ ，其中0表示负类，1表示正类。



Images are 28 x 28 pixels

输入：每一个图像表示为28*28维的向量 $\mathbf{x} \in R^{784}$
学习一个分类器 $f: \mathbf{x} \mapsto \{0,1,2,3,4,5,6,7,8,9\}$

标签	短信内容
0	商业秘密的秘密性那是维系其商业价值和垄断地位的前提条件之一
1	南口阿玛施新春第一批限量春装到店啦 春暖花开淑女裙、冰蓝色公主衫 气质粉小西装、冰丝女王长半裙
0	带给我们大常州一场壮观的视觉盛宴
0	有原因不明的泌尿系统结石等
0	23年从盐城拉回来的麻麻的嫁妆
1	感谢致电杭州萧山全金釜韩国烧烤店，本店位于金城路xxx号。韩式烧烤等，价格实惠、欢迎惠顾【全金釜韩国烧烤店】
1	(长期诚信在本市作各类资格职称 (以及印 /章、牌、等。祥: xxxxxxxxxxxx 李伟%

输入：每一个短信文本
学习一个分类器 $f: \mathbf{x} \mapsto \{\text{正常短信}, \text{垃圾短信}\}$



模型

- 一元线性回归:

- $y = f(x) = w \cdot x + b$
 - 自变量 x 和参数 w 和 b 均为一元变量 (标量)
- $\hat{y}_i = w \cdot x_i + b$

- 多元线性回归:

- $y = f(x) = \mathbf{x} \cdot \mathbf{w}$
 - 在 $\mathbf{x}$ 后面加上一维值恒为1的特征, $\mathbf{x} \leftarrow [\mathbf{x}, 1]$
 - 自变量 $\mathbf{x}$ 为 $d+1$ 维行向量 ($1 \times (d+1)$)
 - 参数 $\mathbf{w}$ 为 d 维列向量 ($(d+1) \times 1$)
- $\hat{y}_i = \mathbf{x}_i \cdot \mathbf{w}$

$$\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,j}, \dots, x_{i,d}, \mathbf{1})$$

- 逻辑回归:

$$- P(y = 1) = g(\mathbf{x} \cdot \mathbf{w}) = \frac{1}{1+e^{-\mathbf{x} \cdot \mathbf{w}}}, \quad \hat{y}_i = \begin{cases} 1 & g(\mathbf{x} \cdot \mathbf{w}) \geq 0.5 \\ 0 & g(\mathbf{x} \cdot \mathbf{w}) < 0.5 \end{cases}$$

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_j \\ \dots \\ w_d \\ w_{d+1} \end{pmatrix}$$



损失函数

- 使用任何一组参数都可以得到一组预测值 $\hat{y}$ ，需要一个标准来对预测结果进行度量：定量化一个目标函数式度量预测结果的好坏。
- 回归问题：
 - MSE(mean square error)均方误差：线性模型应用于训练样本 x_i ，得到预测值 $\hat{y}_i$ ，将预测值和真实值 y_i 之间的平均的平方距离定义为损失函数：

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- N 为训练集样本总数
- 二分类问题：
 - BCE (binary cross entropy) (二元) 交叉熵函数：线性模型应用于训练样本 x_i ，在经过Sigmoid函数计算：

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

- p_i 为模型预测第 i 个样本为正例 ($y_i = 1$)的概率



优化

- 一元线性回归:
 - 对 w 求导:

$$\frac{\partial L(w, b)}{\partial w} = \frac{1}{N} \sum_{i=1}^N 2(wx_i + b - y_i)x_i$$

$$\frac{\partial L(w, b)}{\partial w} = \frac{2}{N} \mathbf{x}^T (w\mathbf{x} + b\mathbf{1} - \mathbf{y})$$

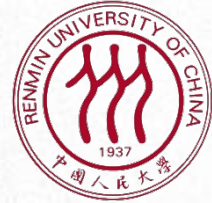
$\mathbf{x}$ 、 $\mathbf{1}$ 、 $\mathbf{y}$ 为 $N \times 1$ 的向量 (列向量)

- 对 b 求导:

$$\frac{\partial L(w, b)}{\partial b} = \frac{2}{N} \sum_{i=1}^N (wx_i + b - y_i) = \frac{2}{N} \sum_{i=1}^N 1 \times (wx_i + b - y_i)$$

$$\frac{\partial L(w, b)}{\partial w} = \frac{2}{N} \mathbf{1}^T (w\mathbf{x} + b\mathbf{1} - \mathbf{y})$$

$\mathbf{x}$ 、 $\mathbf{1}$ 、 $\mathbf{y}$ 为 $N \times 1$ 的向量 (列向量)



优化

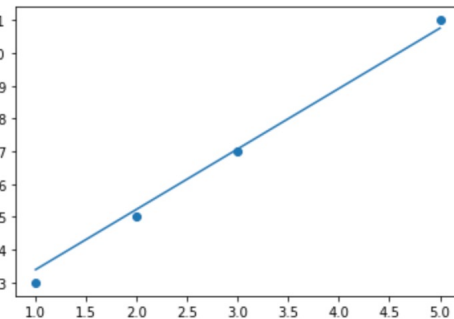
- 一元线性回归：
- 1. 随机选择一组初始参数(w^0, b^0)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$w^{t+1} = w^t - lr \frac{\partial L(w, b)}{\partial w} (w^t)$$

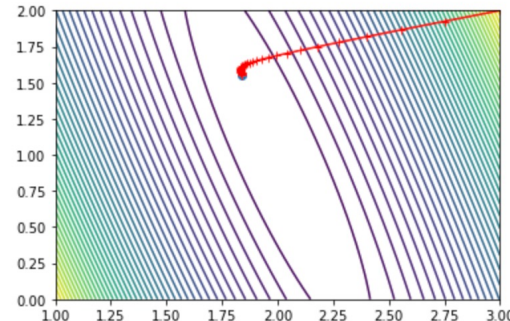
$$b^{t+1} = b^t - lr \frac{\partial L(w, b)}{\partial b} (b^t)$$

- 4. 循环迭代步骤 3，直到前后两次迭代得到的 (w^t, b^t) 差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出 (w^t, b^t) ，即为使得损失函数最小时的参数取值。

#iter: 40: $y=1.8397121239629468*x+1.5565434196318897$



current loss: 0.06960016457557064

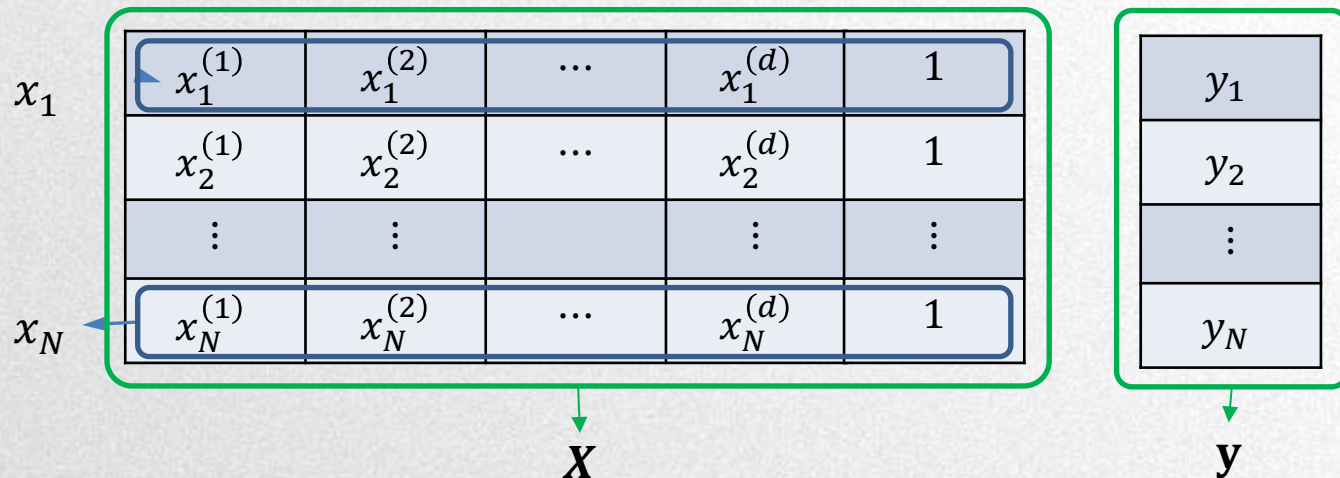


优化

- 多元线性回归：
- Step 1：分别对参数 $\mathbf{w}$ 求导，将结果组织成与 $\mathbf{w}$ 形状相同的数组 $((d+1), 1)$ ：

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = \begin{pmatrix} \frac{2}{N} \sum_{i=1}^N x_{i,1} (\mathbf{x}_i \cdot \mathbf{w} - y_i) \\ \frac{2}{N} \sum_{i=1}^N x_{i,2} (\mathbf{x}_i \cdot \mathbf{w} - y_i) \\ \dots \\ \frac{2}{N} \sum_{i=1}^N x_{i,d+1} (\mathbf{x}_i \cdot \mathbf{w} - y_i) \end{pmatrix} = \frac{2}{N} \sum_{i=1}^N \mathbf{x}_i^T (\mathbf{x}_i \cdot \mathbf{w} - y_i)$$

优化



$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \sum_{i=1}^N \mathbf{x}_i^T (\mathbf{x}_i \cdot \mathbf{w} - y_i), \text{ 其中 } \mathbf{x}_i \text{ 为行向量}$$

- Step 2: 写成矩阵计算的形式:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

注意: 此处 $\mathbf{X}$ 的维度是 $N \times (d+1)$, $\mathbf{w}$ 维度为 $(d+1) \times 1$



优化：梯度下降法

- 1. 随机选择一组初始参数($\mathbf{w}^0$)
- 2. 对于可微函数，梯度方向时函数增长速度最快的方向，梯度的反方向是函数减少最快的方向。计算损失函数在当前参数点处的梯度。
- 3. 从当前参数点向梯度相反的方向移动，移动步长为：

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}(\mathbf{w}^t)$$

- 4. 循环迭代步骤 3，直到前后两次迭代得到的($\mathbf{w}^t$)差值足够小，即参数基本不再变化，说明此时损失函数已经达到局部最小值。
- 5. 输出($\mathbf{w}^t$)，即为使得损失函数最小时的参数取值。



示例：梯度下降法（类的定义和初始化）

```
class MultiLinearRegression(object):  
    def __init__(self, dim_in, learning_rate=0.01, max_iter=100, seed=None):  
        """  
        一元线性回归类的构造函数：  
        参数 学习率: learning_rate  
        参数 最大迭代次数: max_iter  
        参数 seed: 产生随机数的种子  
        从正态分布中采样w的初始值  
        """  
        np.random.seed(seed)  
        self.lr = learning_rate  
        self.max_iter = max_iter  
        self.w = np.random.normal(1, 0.1, [dim_in+1, 1]) # w 的维度为输入维度+1  
        self.loss_arr = []
```




示例：梯度下降法

```
def fit(self, x, y):
```

```
    """
```

```
    类的方法：训练函数
```

```
    参数 自变量: x
```

```
    参数 因变量: y
```

```
    返回每一次迭代后的损失函数
```

```
    """
```

```
    # 首先在x矩阵后面增加一列1
```

```
    x = np.hstack([x, np.ones((x.shape[0], 1))])
```

```
    for i in range(self.max_iter):
```

```
        self.__train_step(x, y)
```

```
        y_pred = self.predict(x)
```

```
        self.loss_arr.append(self.loss(y, y_pred))
```

```
def __train_step(self, x, y):
```

```
    """
```

```
    类的方法：单步迭代，即一次迭代中对梯度进行更新
```

```
    """
```

```
    d_w = self.__calc_gradient(x, y)
```

```
    self.w = self.w - self.lr * d_w
```

```
    return self.w
```

```
def loss(self, y_true, y_pred):
```

```
    """
```

```
    类的方法：计算损失
```

```
    参数 真实因变量: y_true
```

```
    参数 预测因变量: y_pred
```

```
    返回: MSE 损失
```

```
    """
```

```
    return np.mean((y_true - y_pred) ** 2)
```

```
def __calc_gradient(self, X, y):
```

```
    """
```

```
    类的方法：分别计算对w和b的梯度
```

```
    """
```

```
    N = X.shape[0]
```

```
    #diff = (X.dot(self.w) - y)
```

```
    diff = np.matmul(X, self.w) - y
```

```
    #print(type(X.T))
```

```
    grad = np.matmul(X.T, diff) # X.T 转置 x.transpose()
```

```
    d_w = (2 * grad) / N
```

```
    return d_w
```

```
def __f(self, X, w):
```

```
    """
```

```
    类的方法：计算一元线性回归函数在x处的值
```

```
    """
```

```
    return np.matmul(X, w)
```

```
def predict(self, X):
```

```
    """
```

```
    类的方法：预测函数
```

```
    参数：自变量: x
```

```
    返回：对x的回归值
```

```
    """
```

```
    y_prd = self.__f(X, self.w)
```

```
    return y_prd
```



示例：梯度下降法（训练模型）

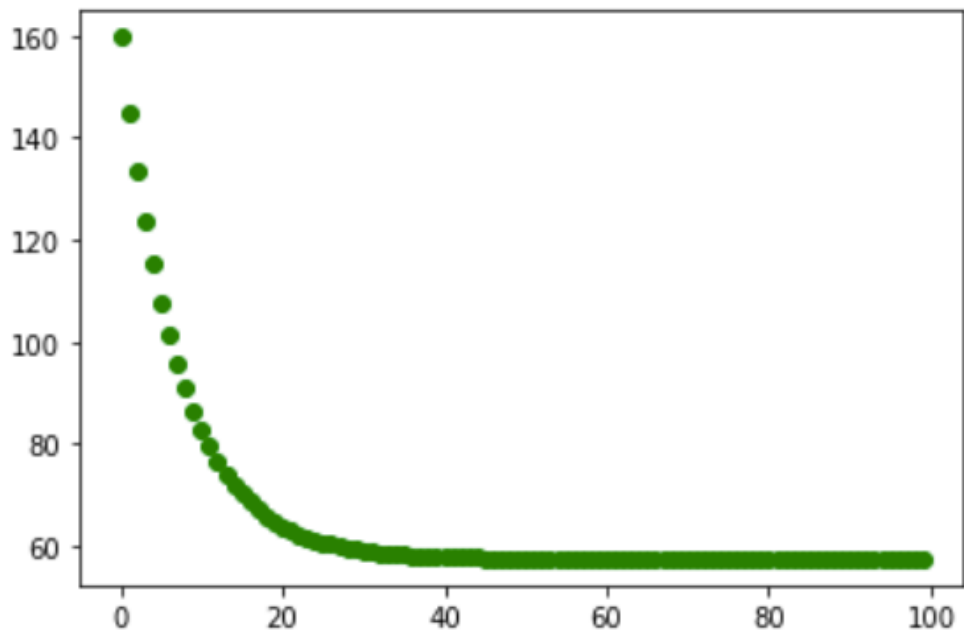
```
# train the liner regression model
regr = LinearRegressionMulti(dim_in, learning_rate=0.01, max_iter=100, seed=0)
regr.fit(x_train, y_train)
print(regr.w)

x_test_aug = np.hstack([x_test, np.ones((x_test.shape[0], 1))])
y_pred = regr.predict(x_test_aug)
res = y_pred - y_test

# plot the evolution of cost
plt.scatter(np.arange(len(regr.loss_arr)), regr.loss_arr, marker='o', c='green')
plt.show()
```

示例：梯度下降法

```
[[ 0.53989083]  
 [-4.89837128]  
 [ 3.30861944]  
 [ 0.78811499]]
```





逻辑回归 (Logistic Regression) 模型

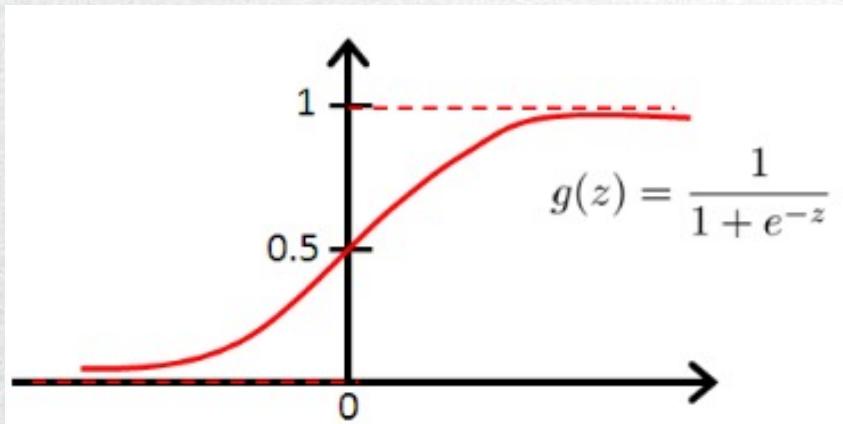
- 二分类：因变量 y 可能属于的两个类分别称为负类和正类，则因变量 $y \in \{0, 1\}$ ，其中0表示负类，1表示正类。
- 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
- Logistic回归：引入非线性变换sigmoid函数 $g(z)$ ，把线性回归的输出值压缩到 $(0, 1)$ 之间：

$$p = g(\mathbf{x}\mathbf{w})$$

$$\hat{y} = 1 \text{ if } p \geq 0.5 \text{ else } 0$$

其中， $\mathbf{w}$ 为模型的参数。

注意：已经在 $\mathbf{x}$ 后添加恒为1的特征，
将参数 b 融入了 $\mathbf{w}$





逻辑回归模型

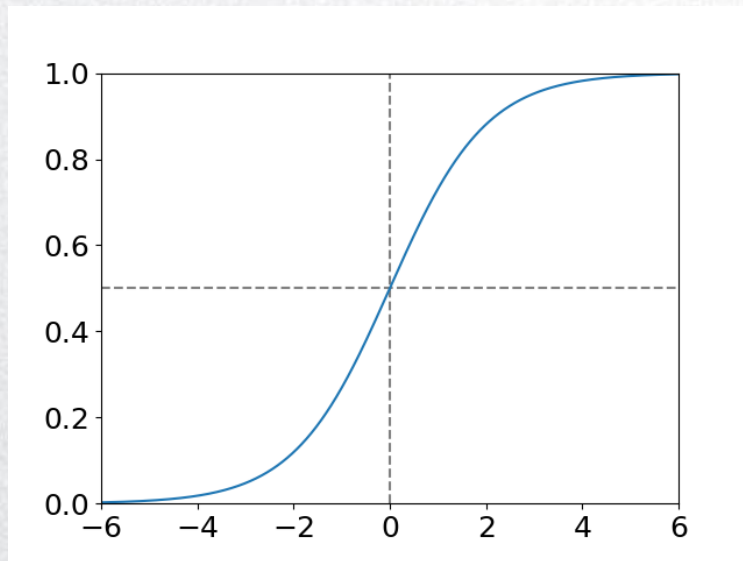
- 什么是“回归”？
 - 预测的是落入某一分类的**概率**
 - $\hat{y} = p$
- 又叫“对数几率”回归
 - 设某事件发生概率为 $p \in [0,1]$
 - 若采用线性回归，则无法保证输出在 $[0,1]$ 范围内
 - 机率比（发生比、优势比）： $\frac{p}{1-p} \geq 0$
 - 取对数，令 $z = \ln \frac{p}{1-p} \in (-\infty, +\infty)$
 - 可以采用线性回归模型拟合： $z = f(\mathbf{x}) = \mathbf{x}\mathbf{w}$

Logistic回归模型

- 逻辑回归模型:

$$xw = z = \ln \frac{p}{1-p} \Rightarrow p = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-(xw)}}$$

- Sigmoid函数 $g(z) = \frac{1}{1+e^{-z}} \in (0,1)$
 - 一般用于表示概率





逻辑回归损失函数的构造

- 什么样的参数 $\mathbf{w}$ 会使得观察到训练数据 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$ 的概率最大?
 - 对于单个训练数据 $(\mathbf{x}_i, y_i)$
 - 如果为正例: $y_i = 1$, 使得 p_i 最大
 - 如果为负例: $y_i = 0$, 使得 $1 - p_i$ 最大

↓

 - 总体而言: 使得 $p_i^{y_i} \times (1 - p_i)^{1-y_i}$ 最大
 - 取对数: 使得 $y_i \ln p_i + (1 - y_i) \ln(1 - p_i)$ 最大
- 对于 N 个训练数据 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$, 使得它们的联合概率越大 (负联合概率最小)

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

训练 (优化)

- 如何计算Logistic回归模型参数关于交叉熵损失函数的梯度?
 - 模型: $p = g(\mathbf{x}\mathbf{w}) = \frac{1}{1+e^{-\mathbf{x}\mathbf{w}}}$
 - 损失函数: $L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$
- 梯度计算:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = ?$$

训练 (优化)

- 如何计算Logistic回归模型参数关于交叉熵损失函数的梯度?
 - 模型: $p = g(\mathbf{x}\mathbf{w}) = \frac{1}{1+e^{-\mathbf{x}\mathbf{w}}}$
 - 损失函数: $L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$
- 梯度计算:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = \sum_{i=1}^N \frac{\partial L}{\partial p_i} \begin{pmatrix} \frac{\partial p_i}{\partial w_1} \\ \frac{\partial p_i}{\partial w_2} \\ \dots \\ \frac{\partial p_i}{\partial w_{d+1}} \end{pmatrix}$$



逻辑回归梯度的具体推导

- 交叉熵损失函数:

$$- L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

- 交叉熵损失函数关于预测值 p_i 的偏导数:

$$\frac{\partial L}{\partial p_i} = -\frac{1}{N} \left(\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right)$$



逻辑回归梯度的具体推导

- Sigmoid函数导数: $p = g(z) = \frac{1}{e^{-z} + 1}$
$$\frac{dp}{dz} = \frac{dg}{dz} = g(z)(1 - g(z)) = p(1 - p)$$



逻辑回归梯度的具体推导

- Sigmoid函数导数: $p = g(z) = \frac{1}{e^{-z} + 1}$
$$\frac{dp}{dz} = \frac{dg}{dz} = g(z)(1 - g(z)) = p(1 - p)$$

$$\frac{\partial p_i}{\partial w^j} = \frac{\partial g(\mathbf{x}_i \mathbf{w})}{\partial w^j} = g(\mathbf{x}_i \mathbf{w})(1 - g(\mathbf{x}_i \mathbf{w})) \frac{\partial (\mathbf{x}_i \mathbf{w})}{\partial w^j} = p_i(1 - p_i)x_i^j$$

逻辑回归梯度的具体推导

- 交叉熵损失函数：

- $L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$

- 交叉熵损失函数关于预测值 p_i 的偏导数：

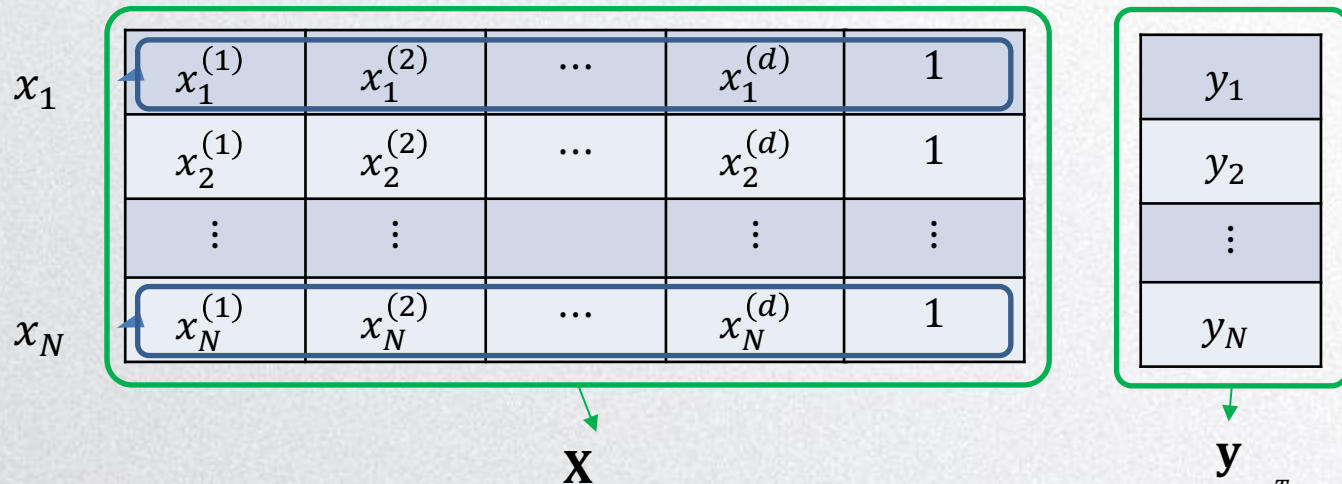
$$\frac{\partial L}{\partial p_i} = -\frac{1}{N} \left(\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right)$$
$$\frac{\partial p_i}{\partial w^j} = \frac{\partial g(\mathbf{x}_i \mathbf{w})}{\partial w^j} = g(\mathbf{x}_i \mathbf{w}) (1 - g(\mathbf{x}_i \mathbf{w})) \frac{\partial (\mathbf{x}_i \mathbf{w})}{\partial w^j} = p_i (1 - p_i) x_i^j$$

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = \sum_{i=1}^N \frac{\partial L}{\partial p_i} \begin{pmatrix} \frac{\partial p_i}{\partial w_1} \\ \frac{\partial p_i}{\partial w_2} \\ \dots \\ \frac{\partial p_i}{\partial w_{d+1}} \end{pmatrix}$$

逻辑回归梯度的具体推导

$$\begin{aligned}
 \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} &= \begin{pmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \dots \\ \frac{\partial L}{\partial w_{d+1}} \end{pmatrix} = \sum_{i=1}^N \frac{\partial L}{\partial p_i} \begin{pmatrix} \frac{\partial p_i}{\partial w_1} \\ \frac{\partial p_i}{\partial w_2} \\ \dots \\ \frac{\partial p_i}{\partial w_{d+1}} \end{pmatrix} = -\frac{1}{N} \sum_{i=1}^N \left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i} \right] \begin{pmatrix} p_i(1-p_i)x_i^1 \\ p_i(1-p_i)x_i^2 \\ \dots \\ p_i(1-p_i)x_i^{d+1} \end{pmatrix} \\
 &= -\frac{1}{N} \sum_{i=1}^N [y_i(1-p_i) - (1-y_i)p_i] \begin{pmatrix} x_i^1 \\ x_i^2 \\ \dots \\ x_i^{d+1} \end{pmatrix} \\
 &= \frac{1}{N} \sum_{i=1}^N (p_i - y_i) \begin{pmatrix} x_i^1 \\ x_i^2 \\ \dots \\ x_i^{d+1} \end{pmatrix}
 \end{aligned}$$

梯度的矩阵形式



$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \left(\frac{1}{N} \sum_{i=1}^N x_i^{(1)} (p_i - y_i), \frac{1}{N} \sum_{i=1}^N x_i^{(2)} (p_i - y_i), \dots, \frac{1}{N} \sum_{i=1}^N x_i^{(d+1)} (p_i - y_i) \right)^T$$

• 矩阵形式:

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

注意: 此处 $\mathbf{X}$ 的维度是 $N \times (d + 1)$, $\mathbf{p} - \mathbf{y} = \begin{bmatrix} p_1 - y_1 \\ \vdots \\ p_N - y_N \end{bmatrix}$ 维度为 $N \times 1$



逻辑回归

- 梯度下降

1. 随机初始化参数 w
2. 计算

$$p = \frac{1}{1 + e^{-(Xw)}}$$

3. 更新参数 w

$$w \leftarrow w - \alpha \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

4. 重复步骤2、3直至收敛

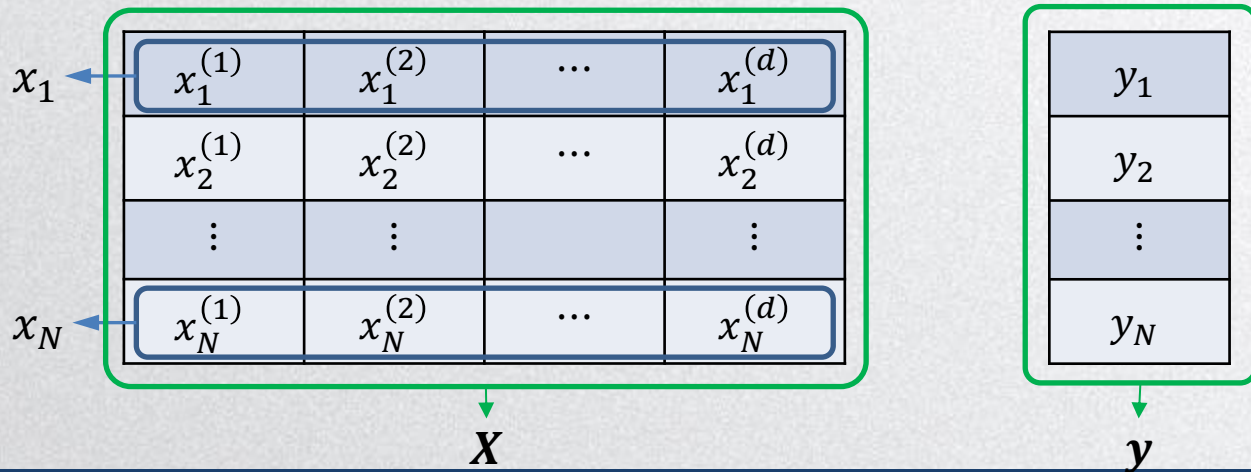


作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 阅读多元线性回归示例代码，理解每个部分的含义
 - 在此基础上进行修改实现逻辑回归代码
 - 注意将计算向量化以提高计算速度，充分利用numpy在矩阵计算上的优势

作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 实现类的方法 `loss = fit(X,Y)` 函数，用于训练
 - 输入：X为N * d维的训练数据，N为训练样本数，d为数据的维数，即X是N个d维数据点排成的矩阵；Y为N*1维的训练数据真实类别号，即N个类别号排成的向量；
 - 输出：loss为列表，长度为训练轮数，每个元素为对应轮的损失函数值。





作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 实现类的方法 `loss = fit(X,Y)` 函数，用于训练
 - 实现步骤：
 1. 扩充 x_i ，增加常数1在末尾，并将所有 x_i 组合成 $\mathbf{X}$ ，将所有 y_i 组合成 $\mathbf{y}$
 2. 计算损失函数值，加入到loss列表结尾
 3. 计算损失函数对参数 $\mathbf{W}$ 的梯度
 4. 更新参数 $\mathbf{W}$
 5. 重复上述过程直至收敛



作业：用numpy实现逻辑回归

- 用numpy实现Logistic Regression类，用梯度下降法训练
 - 实现类的方法 `y_pred, y_pred_label = predict(X)` 函数，用于测试
 - 输入：X为N * d维的测试数据，N为测试样本数；
 - 输出：y_pred维数为N*1，为模型的预测（回归）值，即属于类别1的概率；y_pred_label维数为N*1，为根据回归值得到的预测类别号，只能取0或1
 - y_pred_label的获得方法：若相应的y_pred小于0.5，则y_pred_label为0，否则为1



作业：用numpy实现逻辑回归

```
import numpy as np

class LogisticRegression(object):
    def __init__(self, dim, learning_rate=0.01, max_iter=100, seed=None):
        np.random.seed(seed)
        self.lr = learning_rate
        self.max_iter = max_iter # 定义学习率和训练轮数
        # 可在此处补充类的属性

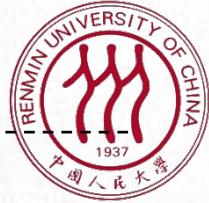
    def fit(self, X, Y):
        # 请在此处补充类的方法：训练函数，返回每轮loss的列表
        return loss

    def predict(self, X):
        # 请在此处补充类的方法：测试函数，返回对应X的预测值和预测类别列表
        return Y_pred, Y_pred_label
```




Python 0X 人工智能导论

提纲



- □ 人工智能的发展历史
- □ 人工智能的重要应用
- □ 人工智能的重要进展
- □ 展望与总结

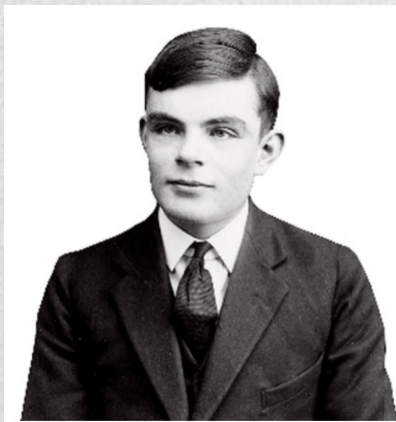


人工智能的发展历史

- 人工智能的诞生和图灵测试
- 初期时代
- 知识时代
- 机器学习时代

人工智能的发展历史

- 人工智能的诞生 (20世纪40 ~ 50年代)
 - 1950年, 图灵测试
 - 1954年, 第一台可编程机器人诞生



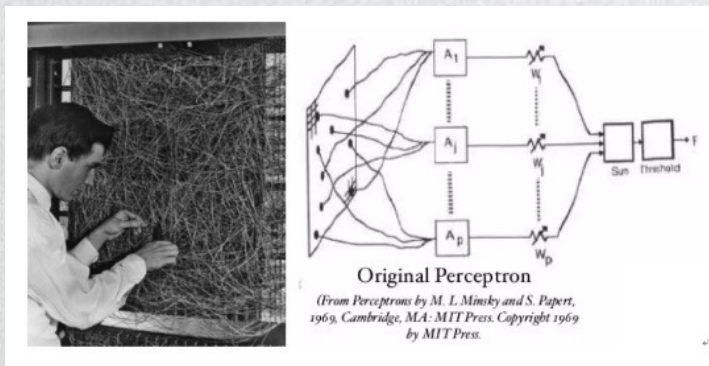
人工智能的诞生：达特茅斯会议

- 1956年，第一次人工智能研讨会
(Dartmouth Summer Research Project on Artificial Intelligence)
 - 在Proposal中首次出现了Artificial Intelligence 一词
- 会上讨论了人工智能研究的七个议题：
 - 自动计算机 (可编程计算机)
 - 如何对计算机进行编程以使用语言
 - **神经网络**
 - 计算规模理论 (计算复杂性)
 - **自我改进 (机器学习)**
 - 抽象
 - **随机性与创造性(AIGC?)**



● 人工智能的发展历史：初期时代（1956-70'）

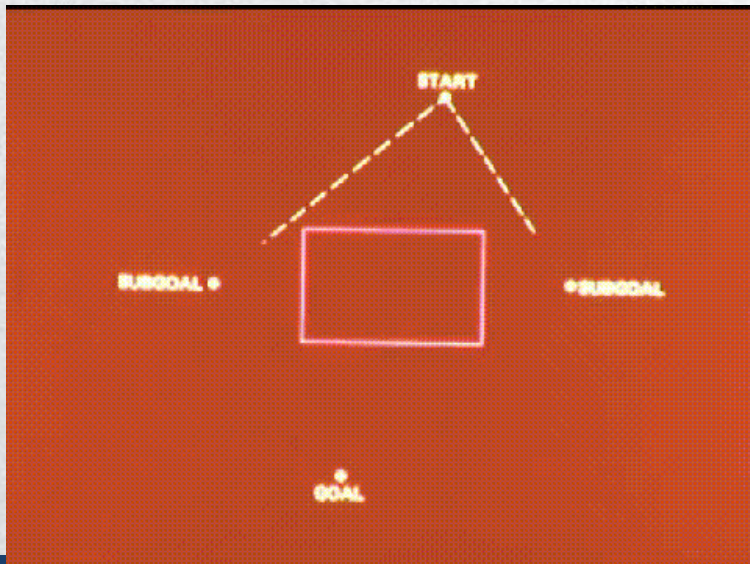
- 1957: Frank Rosenblatt提出感知机模型，人工神经网络的雏形开始出现
- 1959: Arthur Samuel（亚瑟·塞缪尔）完成了最高的机器学习实验，让程序学会了纯粹通过自己与自己玩来学习跳棋游戏，这是一项惊人的壮举





● 人工智能的发展历史：初期时代（1956-70'）

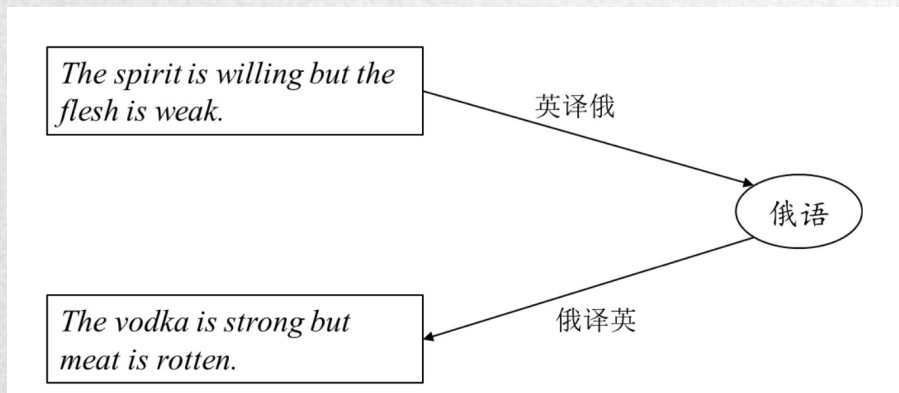
- 人工智能的第一次黄金时代（20世纪50～70年代）
 - 1966年~1972年：首台人工智能机器人Shakey诞生
 - 1966年：世界上第一个聊天机器人ELIZA发布
 - 1968年：计算机鼠标发明





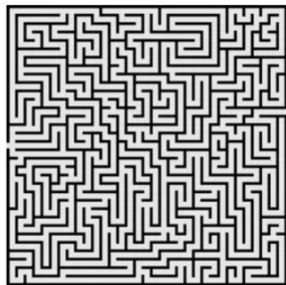
● 人工智能的发展历史：初期时代（1956-70'）

- 取得了很多成果，但对困难认识不足，很快陷入困境
- 以机器翻译为例：

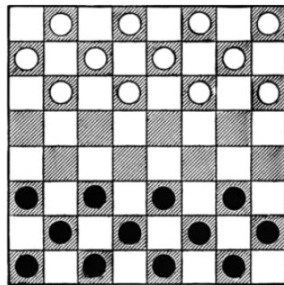


人工智能的发展历史

- 人工智能的第一次低谷（20世纪70 ~ 80年代）
 - 计算机有限的内存和处理速度不足以解决实际的人工智能问题
 - 要求程序对这个世界具有儿童水平的认识很难实现
 - 对于人工智能研究的资助逐渐减少



每多一个路口
可能路线计算量翻倍



西洋跳棋每步
可能棋局 10^{20}



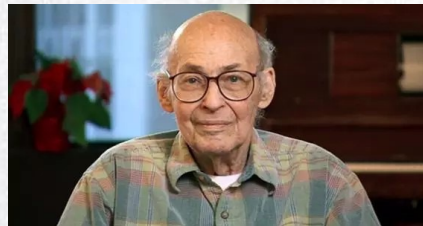
国际象棋每步
可能棋局 10^{40}

人工智能的发展历史

- 人工智能的第一次低谷（20世纪70～80年代）
 - 1966：美国科学院语言自动处理咨询委员会(ALPAC) 经过两年的研究，公布了一份报告否定了机器翻译的可行性
 - 1969：马文·明斯基撰写了名为《感知器》一书，指出了当时新兴神经网络领域的关键问题和缺陷
 - 1973：小丘报告（Lighthill report）怀疑人工智能是否有必要做为是一门独立学科存在



John R. Pierce (ALPAC主席)



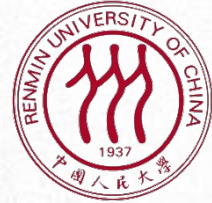
马文·明斯基 (Marvin Minsky)

上述报告把人工智能打入“冷宫”并经历了几乎十年“严冬”



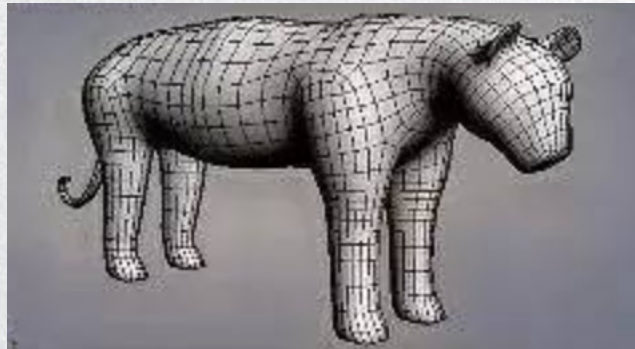
● 人工智能的发展历史：知识时代（80' -90' ）

- 以专家系统为代表
 - 采用（基于规则的）知识表示和知识推理技术来模拟通常由领域**专家**才能解决的复杂问题
 - 爱德华·费根鲍姆
 - 开发了世界上第一个专家系统 DENDRAL,帮助化学家判断某待定物质的分子结构
 - MYCIN,帮助医生对血液感染者进行诊断和药物治疗



● 人工智能的发展历史：知识时代（80' -90' ）

- 人工智能的繁荣期（1980年~1987年）
 - 1984年：启动Cyc（大百科全书）项目
 - 1986年：3D打印机问世
 - 1986年：反向传播算法（Backpropagation algorithm）出现



● 人工智能的发展历史：知识时代（80' -90'）

- 专家系统的出现让人工智能走向了实用
 - XCON：第一个实用商用并带来经济效益的专家系统，帮助DEC公司为计算机系统配置订单
 - 在国际象棋上战胜卡斯帕罗夫，在中国象棋上战胜柳大华



人工智能的发展历史

- 人工智能的第二次冬天 (1987年~1993年)
 - 专家系统难以升级扩展, 鲁棒性不够, 导致高昂的维护成本
 - Lisp机器和Thinking Machines都做不出来
 - 神经网络效果未能超过统计的方法, 资源消耗大, 无法解决大规模问题
 - 日本第五代计算机计划失败
 - 政府投入缩减
 - PC普及, 大型计算机受到冲击

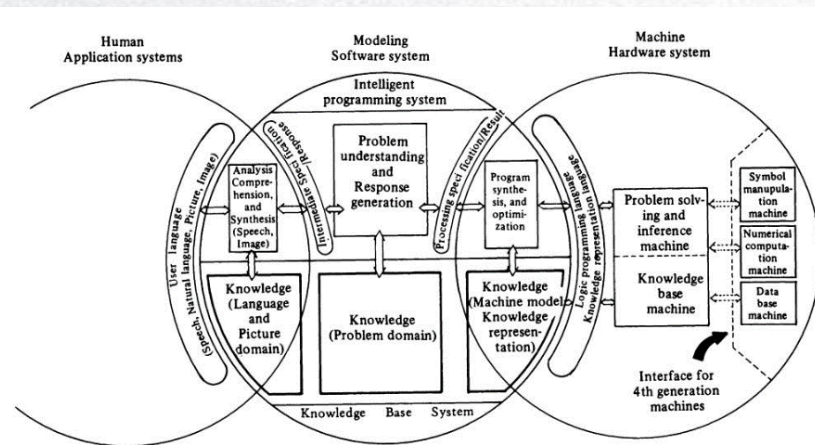


Fig. 4-1 Conceptual diagram of a fifth generation computer system as viewed from the standpoint of programming

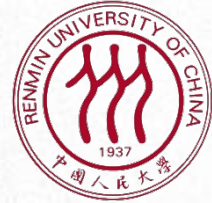
人工智能的发展历史

- 人工智能的发展历史

- 人工智能的第二次冬天 (1987年~1993年)

- “AI之冬”一词由经历过1974年经费削减的研究者们创造出来。
 - 专家系统的实用性仅仅局限于某些特定情景
 - 美国国防部高级研究计划局 (DARPA) 认为人工智能并非 “下一个浪潮”





● 人工智能的发展历史：机器学习时代（90' -现在）

- 专家系统的问题：获取专家知识可能非常困难
 - 如何获取让机器能骑自行车的知识？
- 解决方法：让机器从数据中自动的学习
- 统计机器学习时代
 - 人工提取特征，机器自动学习参数
 - 被用于语音、图像、文本、机器人等各个领域
 - 基于AI技术的搜索引擎获得巨大的成功（Google、Yahoo!等）



● 人工智能的发展历史：机器学习时代（90' -现在）

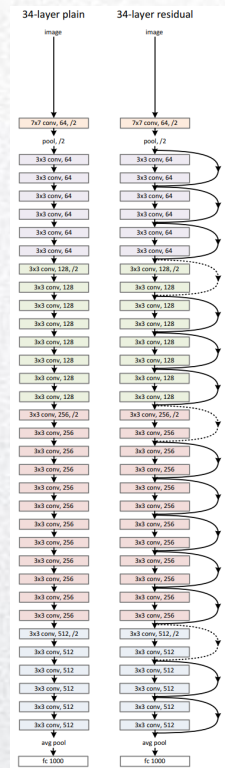
- 深度学习崛起：
 - 2006：Geoffrey Hinton发表Science论文



G. E. Hinton, S. Osindero, and Y. Teh, “A fast learning algorithm for deep belief nets,” Neural Computation, vol. 18, pp. 1527–1554, 2006.

G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” Science, vol. 313, no. 5786, pp. 504-507, July 2006.

- 神经网络借深度学习之名再度登场
- 2009：ImageNet数据集发布，2010年举行首次ImageNet竞赛

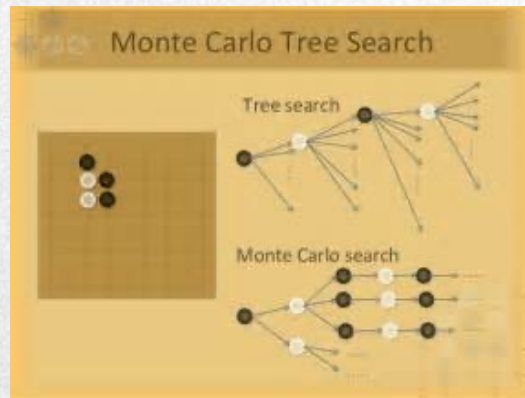


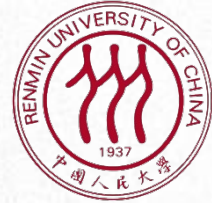
深度神经网络

● 人工智能的发展历史：机器学习时代（90' -现在）

● 深度学习崛起：

- 2016年3月，DeepMind开发的人工智能围棋程序AlphaGo击败职业棋手李世石
- 通过**深度强化学习模型**估算每一步落子的胜率





● 人工智能的发展历史：机器学习时代（90' -现在）

- 深度学习崛起：
 - 大数据+深度网络+大规模计算
 - 规模大到甚至引发了对环境保护的担忧
 - 做“有温度的人工智能”

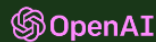
	Date of original paper	Energy consumption (kWh)	Carbon footprint (lbs of CO2e)	Cloud compute cost (USD)
Transformer (65M parameters)	Jun, 2017	27	26	\$41-\$140
Transformer (213M parameters)	Jun, 2017	201	192	\$289-\$981
ELMo	Feb, 2018	275	262	\$433-\$1,472
BERT (110M parameters)	Oct, 2018	1,507	1,438	\$3,751-\$12,571
Transformer (213M parameters) w/ neural architecture search	Jan, 2019	656,347	626,155	\$942,973-\$3,201,722
GPT-2	Feb, 2019	-	-	\$12,902-\$43,008

Note: Because of a lack of power draw data on GPT-2's training hardware, the researchers weren't able to calculate its carbon footprint.

Table: MIT Technology Review • Source: Strubell et al. • Created with [Datawrapper](#)

GPT系列模型的规模

- 2017年, 谷歌提出Transformer
- 2018年, OpenAI提出GPT (1亿+参数)
- 2019年, GPT2 (15亿参数)
- 2020年, GPT3 (**1750亿参数**)
- 2021年, CodeX (基于GPT3, 赋予了代码和思维链能力)
- 2021年, WebGPT (搜索能力)
- 2022年2月, InstructGPT (**理解人类指令**)
- 2022年11月, ChatGPT (**对话能力**)
- 2023年3月, GPT-4 (多模态, 长上下文, **通用人工智能?**)
- 2024年12月, o1 (**推理能力**)



ChatGPT: Optimizing Language Models for Dialogue

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer followup questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests. ChatGPT is a sibling model to InstructGPT, which is trained to follow an instruction in a prompt and provide a detailed response.

TRY CHATGPT ↗



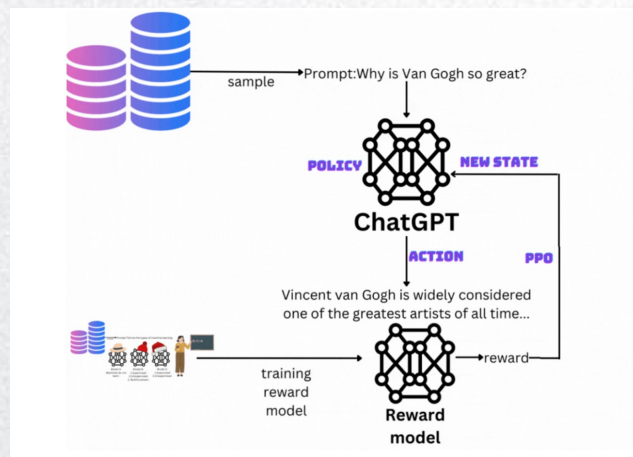
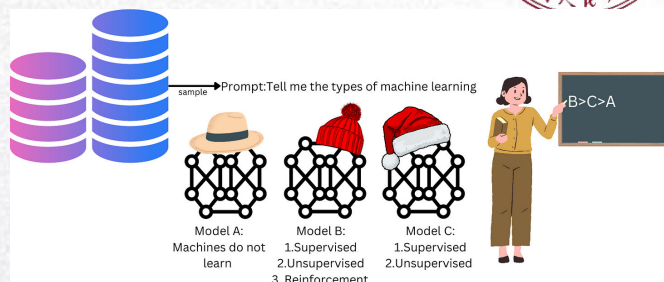
GPT



- GPT (Generative pre-trained transformer)
 - 生成式预训练的Transformer
- GPT (2018): 包含1.17亿个参数, 训练数据5GB
- GPT-2 (2019): 包含15亿个参数 –单纯的自监督, 删除了下游任务, 训练数据40GB
- GPT-3(2020): 拥有1750亿个参数 (100X) –单纯的自监督
 - 数字化书籍和各种网页链接。
 - 不仅包括新闻文章、食谱和诗歌之类的内容, 还包括程序代码、科幻小说、宗教预言等各种你可以想象到的任何文字。
 - 训练数据45T
- 通过预测下一个词 (next token prediction) , 掌握大量人类知识

InstructGPT和ChatGPT

- 通过有监督微调 and 强化学习让模型掌握遵循指令，进行对话的能力
- SFT (Supervised Fine-Tuning) :
 - 收集大量指令，并且聘用了专门的合同工撰写标准答案用于训练模型
 - “背答案”
- RLHF (Reinforcement Learning with Human Feedback) :
 - 使用SFT训练后的模型得到多个答案，请人按照大难质量排序，训练出一个打分模型
 - “改作文” -> “训练模型来改作文”





推理模型

- 从思维链（Chain of thoughts）到DeepSeek R1
 - 通过允许模型生成更长的内容，提升模型的思考和推理能力

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✖

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✔

Let's think step by step

Kojima, T., Gu, S.S., Reid, M., Matsuo, Y. and Iwasawa, Y. Large language models are zero-shot reasoners. NeurIPS 2022.

Question in standard QA format

Q: I have 3 apples, my dad has 2 more apples than me, how many apples do we have in total?
A:

Language model

Decoding step 0

top-1: 5
top-2: I
top-3: We
top-4: You
top-5: The

Continue greedy decoding

5 apples ✖
I have 3 apples, my dad has 2 more apples than me, so he has 5 apples. 3+5=8. We have 8 apples in total. ✔
We have 5 apples in total. ✖
You have 3 apples, your dad has 2 more apples than you, so he has 5 apples. 3+5=8. You have 8 apples in total. ✔
The answer is 5. ✖

uncertain certain

我是一个25年的高考考生，选科物化生，您对我报什么专业有什么建议么？

选择专业时，建议综合考虑个人兴趣、学科优势、职业前景等因素。以下是一些适合物化生组合的专业方向：

1. 理工类专业

- 计算机科学与技术：就业前景广阔，适合对编程、算法感兴趣的学生。
- 电子信息工程：涉及电子技术和信息处理，适合对硬件、通信感兴趣的学生。
- 人工智能：未来热门领域，适合对机器学习、数据分析感兴趣的学生。
- 自动化：涉及控制系统和机器人技术，适合对工程和技术感兴趣的学生。

2. 医学与生命科学

- 临床医学：适合有志于从医的学生，学制较长但职业稳定。
- 生物医学工程：结合生物学和工程学，适合对医疗设备研发感兴趣的学生。
- 药学：涉及药物研发，适合对化学和生物学感兴趣的学生。
- 生物技术：涉及基因工程、生物制药感兴趣的学生。

3. 基础科学

- 物理学：适合对理论物理感兴趣的学生，未来可从事科研或教育。
- 化学：适合对材料科学、环境科学感兴趣的学生。
- 生物学：适合对生命科学、生态学感兴趣的学生。

4. 交叉学科

- 环境科学与工程：适合对环境保护、可持续发展感兴趣的学生。
- 材料科学与工程：涉及新材料的研发，适合对化学和物理感兴趣的学生。
- 能源与动力工程：适合对能源技术、新能源开发感兴趣的学生。

您 巴恩康普 (用时 25 秒)

好的，我现在需要帮用户解答关于选专业的问题。用户是选物化生的高考考生，已经25岁了，可能刚通过或者有其他经历，但用户没有具体说明。首先，我要回顾之前的对话历史，确保不会重复信息，同时提供深入的见解。

上一次回复中，我已经列出了理工类、医学与生命科学、基础科学、交叉学科、经济管理类和教育等方向，并给出了一些建议。现在用户再次提问，可能希望得到更详细的分析，或者之前的回答没有完全满足他们的需求。我需要考虑用户可能的潜在需求，比如职业前景、个人兴趣匹配、薪资因素等。

用户25岁，可能对未来的职业发展有更迫切的考虑，或者希望选择能快速就业的专业。这时候需要强调就业前景好、薪资较高的领域，比如计算机、人工智能、电子信息工程等。同时，考虑到年龄，可能希望避免学制过长的专业，比如临床医学（通常需要3年本科加3年规培），但用户如果对此有强烈兴趣，也可以考虑。

另外，用户选科物化生，这三门都是基础理科，适合报考理工科和医学相关专业。需要结合当前的社会趋势，比如新能源、人工智能、生物科技等领域的发展，推荐有潜力的专业。

可能用户还希望了解跨学科的选择，比如金融工程、数据科学。这些虽然属于经济管理类，但需要较强的数理基础。物化生背景的学生也有优势，需要强调这些领域的交叉性和应用性。

同时，用户可能需要考虑学习强度和难度，比如临床医学学习压力大，计算机需要持续学习，而师范类则相对稳定但竞争也可能激烈。需要帮用户评估自己的抗压能力和兴趣来源。

还要考虑用户的个人兴趣，如果用户对特定领域有浓厚兴趣，应该优先考虑。但用户没有提供更多背景信息，所以需要保持建议的广泛性，并鼓励用户进一步提供信息以便更精准推荐。

最后，会结合地域和薪资，分点列出不同方向，强调潜力、薪资、个人兴趣的结合，并提醒结合自身情况做决定。可能还需要提醒考研、就业市场的现状，比如某些专业需要深造才能有更好的发展，用户需要权衡利弊。

现在需要将这些思考整理成自然的中文回复，确保涵盖各个可能的角度，同时避免重复之前的内容，提供新的见解或更详细的建议。

作为选物化生的考生，您的专业选择范围非常广泛。以下是结合当前就业前景、学科特点以及个人发展可能性的具体建议，分为“高潜力赛道”“稳定赛道”和“兴趣导向赛道”三类，供您参考：

一、高潜力赛道（就业前景好/薪资竞争力强）

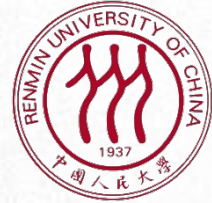
1. 人工智能/大数据/计算机类

- 推荐理由：数字经济时代核心领域，与物化生的数理逻辑能力高度契合。
- 就业方向：算法工程师、数据科学家、AI产品经理（起薪普遍15-30万/年）。
- 注意点：需持续学习编程（Python、C++等）和数学建模能力，适合抗压能力强、对技术敏感的学生。

2. 微电子/集成电路（芯片方向）

- 推荐理由：国家战略重点产业，人才缺口巨大，物理和化学基础是核心优势。
- 就业方向：集成电路设计工程师、半导体工艺工程师（硕士毕业年薪可达20-40万）。

未来展望



Planning for AGI and beyond

Our mission is to ensure that artificial general intelligence—AI systems that are generally smarter than humans—benefits all of humanity.



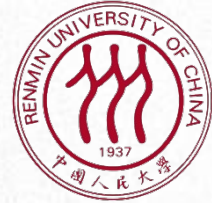
Justin Jay Wang • DALL·E

February 24, 2023

Authors
Sam Altman ↓

Safety & Alignment

Successfully transitioning to a world with **superintelligence** is perhaps the most important—and hopeful, and scary—project in human history. Success is far from guaranteed, and the stakes (boundless downside and boundless upside) will hopefully **unite all of us**.



谢谢！